

Fragment Merging for Nested Pipeline Code Generation in `ssql`

Ross Cartlidge December 2025

Abstract

We present a novel approach to code generation from Unix shell pipelines that supports nested command substitution. The `ssql` CLI tool allows users to prototype data processing pipelines interactively, then generate optimized Go programs from those pipelines. A key challenge arises when pipelines contain process substitution (e.g., `<(cmd)`) for operations like joins and unions that require multiple input sources. We describe a fragment-based code generation architecture that detects nested pipelines, merges their code fragments with automatic variable renaming, and performs dependency graph analysis to produce correct, compilable output. The resulting system enables a seamless transition from interactive prototyping to production-ready compiled programs.

1. Introduction

Data processing pipelines are commonly expressed as chains of Unix commands connected by pipes. This paradigm offers excellent composability and rapid prototyping, but interpreted execution incurs significant overhead compared to compiled programs. The `ssql` project addresses this by generating Go source code from CLI pipelines, achieving 10-100x performance improvements.

However, certain operations require multiple input streams. SQL-style joins, for example, combine records from two sources based on matching keys. In Unix shells, this is naturally expressed using process substitution:

```
ssql from users.csv | ssql join <(ssql from orders.csv) -on user_id
```

The challenge is that when code generation mode is enabled, *both* the outer pipeline and the inner substituted command must emit code fragments rather than data. These fragments must be correctly merged into a single coherent program.

This paper describes our solution: a fragment-based architecture with automatic variable renaming and dependency graph analysis.

2. Background

2.1 The `ssql` Pipeline Model

`ssql` commands communicate via JSONL (JSON Lines) format on `stdin/stdout`. Each command reads records, transforms them, and writes results. This enables arbitrary composition:

```
ssql from data.csv | ssql where -where age gt 18 | ssql sort -by name
```

2.2 Code Generation Mode

When the environment variable `SSQLGO=1` is set, commands emit *code fragments* instead of executing. A code fragment is a JSON object containing:

- **type**: Fragment category (`init`, `stmt`, or `final`)
- **var**: Output variable name
- **input**: Input variable name (dependency)
- **code**: Go source code
- **imports**: Required import paths
- **command**: Original CLI command (for documentation)

The `generate-go` command assembles fragments into a complete Go program.

2.3 Process Substitution

Bash process substitution `<(command)` executes `command` and provides its output as a file descriptor path (e.g., `/dev/fd/63`). This allows commands expecting file arguments to read from dynamic sources.

3. The Problem

Consider a join operation with a filtered secondary source:

```
ssql from users.csv | ssql join <(ssql from orders.csv | ssql where -where status eq active) -on id
```

In code generation mode, we face several challenges:

1. **Nested Fragment Streams:** The inner pipeline `ssql from orders.csv | ssql where ...` produces its own fragment stream, which appears at the `/dev/fd/N` path.
2. **Variable Collisions:** Both pipelines independently name their variables (e.g., `records`, `filtered`). Merging them naively causes naming conflicts.
3. **Dependency Ordering:** The generated code must execute side computations (the filtered orders) before the main pipeline can reference their results.
4. **Graph Structure:** The final program has a tree structure, not a linear chain. The assembler must recognize which fragments belong to the main pipeline versus side computations.

4. Solution Architecture

4.1 Fragment Detection and Reading

When a join or union command receives a `/dev/fd/N` path in generation mode, it attempts to read code fragments from that path rather than treating it as data:

```
if strings.HasPrefix(file, "/dev/fd/") {
    fragments, err := lib.ReadCodeFragmentsFromFile(file)
    if err == nil && len(fragments) > 0 {
        // Process as code fragments
    }
}
```

This auto-detection allows the same command implementation to handle both execution mode (reading JSONL data) and generation mode (reading code fragments).

4.2 Variable Renaming

To prevent collisions, we rename all variables in secondary fragment chains. The renaming strategy assigns:

- A unique prefix to intermediate variables (e.g., `right_` for joins, `union1_` for unions)
- A target name for the final output variable (e.g., `rightRecords`)

```
varRename := make(map[string]string)
for i, frag := range fragments {
    if i == len(fragments)-1 {
        varRename[frag.Var] = "rightRecords" // Final output
    } else {
        varRename[frag.Var] = "right_" + frag.Var // Intermediate
    }
}
```

The renaming is applied to: 1. The fragment's `Var` field 2. The fragment's `Input` field 3. Variable references within the code string

4.3 Dependency Graph Analysis

The assembler must distinguish main pipeline fragments from side computations. We construct a dependency graph by tracing each fragment's `Input` field:

```
tracesToMain := func(varName string) bool {
    current := varName
    for {
        if current == mainPipelineRoot {
            return true
        }
        if current == "" {
            return false // Reached a different root
        }
        frag := varProducers[current]
        current = frag.Input
    }
}
```

Fragments that trace back to the first `init` fragment belong to the main pipeline. Others are side computations that must be emitted before the main `Chain()` call.

4.4 Code Assembly

The final assembly produces:

1. All `init` fragments (data source setup)
2. Side computation `stmt` fragments (standalone statements)
3. Main pipeline `stmt` fragments (combined into `Chain()` if multiple)
4. `final` fragments (output operations)

5. Example

5.1 Input Data

users.csv:

```
id,name,department
1,Alice,Engineering
2,Bob,Sales
3,Carol,Engineering
4,Dave,Marketing
```

orders.csv:

```
id,user_id,amount,status
101,1,500,active
102,2,300,cancelled
103,1,750,active
104,3,200,active
105,4,100,cancelled
```

5.2 Pipeline

We want to find Engineering employees with their active orders over \$400:

```
ssql from users.csv \
| ssql where -where department eq Engineering \
| ssql join <(ssql from orders.csv \
```

```

| ssql where -where status eq active \
| ssql where -where amount gt 400) \
-left-field id -right-field user_id \
| ssql include name amount

```

5.3 Execution Output

```

{"amount":500,"name":"Alice"}
{"amount":750,"name":"Alice"}

```

5.4 Generated Code

```

export SSQLGO=1
ssql from users.csv \
| ssql where -where department eq Engineering \
| ssql join <(ssql from orders.csv \
| ssql where -where status eq active \
| ssql where -where amount gt 400) \
-left-field id -right-field user_id \
| ssql include name amount \
| ssql generate-go

```

Produces:

```

package main

import (
    "fmt"
    "os"
    "github.com/rosscartlidge/ssql/v3"
)

func main() {
    // Main pipeline: read users
    records, err := ssql.ReadCSV("users.csv")
    if err != nil {
        fmt.Fprintf(os.Stderr, "Error: %v\n", fmt.Errorf("reading CSV: %w", err))
        os.Exit(1)
    }

    // Side computation: read and filter orders (variables get unique names)
    right_records_0, err := ssql.ReadCSV("orders.csv")
    if err != nil {
        fmt.Fprintf(os.Stderr, "Error: %v\n", fmt.Errorf("reading CSV: %w", err))
        os.Exit(1)
    }

    right_filtered_1 := ssql.Where(func(r ssql.Record) bool {
        return ssql.GetOr(r, "status", "") == "active"
    })(right_records_0)
    rightRecords := ssql.Where(func(r ssql.Record) bool {
        return ssql.GetOr(r, "amount", float64(0)) > 400
    })(right_filtered_1)

    // Main pipeline: filter, join, project
    included := ssql.Chain(
        ssql.Where(func(r ssql.Record) bool {

```

```

        return ssql.GetOr(r, "department", "") == "Engineering"
    }},
    ssql.InnerJoin(rightRecords, ssql.OnCondition(func(left, right ssql.Record) bool {
        leftVal, leftOk := ssql.Get[any](left, "id")
        rightVal, rightOk := ssql.Get[any](right, "user_id")
        if !leftOk || !rightOk {
            return false
        }
        return fmt.Sprintf("%v", leftVal) == fmt.Sprintf("%v", rightVal)
    })),
    ssql.Include("name", "amount"),
)(records)

// Output
if err := ssql.WriteJSONToWriter(included, os.Stdout); err != nil {
    fmt.Fprintf(os.Stderr, "Error writing output: %v\n", err)
    os.Exit(1)
}
}

```

Note how: - Secondary pipeline variables get unique indexed names (`right_records_0`, `right_filtered_1`)
 - The final output of the side computation becomes `rightRecords` - Side computations appear before the main `Chain()` call - The `rightRecords` variable is referenced in the join within the main pipeline

6. Complexity Analysis

Let n be the total number of fragments and d be the maximum nesting depth.

- **Fragment reading:** $O(n)$ - each fragment is read once
- **Variable renaming:** $O(n * k)$ where k is average code length - string replacements
- **Dependency tracing:** $O(n * d)$ - each fragment traced to its root
- **Code assembly:** $O(n)$ - single pass over categorized fragments

The overall complexity is $O(n * \max(k, d))$, which is effectively linear for typical pipelines.

7. Limitations and Future Work

7.1 Current Limitations

1. **String-based renaming:** Variable renaming uses string replacement, which could theoretically cause issues if variable names appear as substrings in other contexts. A proper AST-based approach would be more robust.
2. **Single nesting level:** While the architecture supports arbitrary nesting in principle, testing has focused on single-level process substitution.
3. **Fragment ordering:** Fragments from secondary pipelines must arrive in dependency order. This is guaranteed by the streaming nature of pipes but could be problematic with concurrent execution.

7.2 Future Directions

1. **Parallel secondary pipelines:** Multiple process substitutions could be read concurrently.
2. **Optimization passes:** The generated code could be optimized (e.g., fusing adjacent filters).
3. **Error context:** Generated code could include source location information for better debugging.

8. Related Work

The concept of generating compiled code from interpreted pipelines has precedents in query compilation for databases (Neumann, 2011) and JIT compilation in data processing systems (Armbrust et al., 2015). Our contribution is applying these ideas to Unix shell pipelines with support for the process substitution idiom.

9. Conclusion

We have presented a practical solution for generating Go code from Unix pipelines containing process substitution. The key innovations are:

1. **Auto-detection** of code fragments versus data at `/dev/fd/N` paths
2. **Systematic variable renaming** with prefixes to prevent collisions
3. **Dependency graph analysis** to separate main pipeline from side computations
4. **Correct ordering** of generated code to satisfy data dependencies

This enables users to prototype complex multi-source data pipelines interactively, then generate production-ready compiled programs with a single command change.

References

- Armbrust, M., et al. (2015). Spark SQL: Relational Data Processing in Spark. *SIGMOD*.
- Neumann, T. (2011). Efficiently Compiling Efficient Query Plans for Modern Hardware. *VLDB*.

The `ssql` project is available at <https://github.com/rosscartlidge/ssql>